



Complemento Práctico

CRUD y Transacciones: El Control Total

Ya sabemos conectarnos a bases de datos. Ahora toca trabajar con ellas de forma profesional. En esta actividad aprenderemos las 4 operaciones fundamentales del desarrollo de software (CRUD: Create, Read, Update, Delete), pero con un enfoque que va mucho más allá de simplemente enviar comandos SQL. Se trata de hacerlo de forma segura, eficiente y consistente.

¿Sabías que concatenar Strings en una consulta SQL es la vulnerabilidad número 1 en la historia de la web? Aquí aprenderás a evitarla usando PreparedStatement, una técnica que no solo protege tu aplicación de ataques maliciosos, sino que también mejora el rendimiento de tus consultas.

Además, entraremos en el fascinante terreno de las Transacciones: aprenderás cómo agrupar varias operaciones para que se ejecuten todas juntas o ninguna, garantizando que nunca queden datos a medias. Este concepto, conocido como atomicidad, es fundamental para mantener la integridad de los datos en aplicaciones empresariales.

Contenido

Seguridad ante Todo

PreparedStatement vs. Concatenación: La diferencia entre una aplicación vulnerable y una protegida contra SQL Injection

Lectura de Datos

Domina el ResultSet y entiende cómo funciona el cursor para evitar errores comunes al recuperar información

Transacciones

El botón de "Deshacer" en bases de datos: aprende a usar commit y rollback para garantizar consistencia

Buenas Prácticas

Por qué debemos restaurar el AutoCommit al final y cómo evitar problemas con pools de conexiones

La Vacuna contra SQL Injection

❏ **⚠ PELIGRO:** En tutoriales antiguos verás código que une variables con el símbolo +. ¡Esto puede destruir tu base de datos!

En muchos tutoriales viejos y código legacy encontrarás consultas SQL construidas de esta manera peligrosa:

```
String sql = "INSERT INTO usuarios VALUES (" + nombre + ")";  
Statement stmt = con.createStatement();  
stmt.executeUpdate(sql);
```

¡ESTO ES UN PELIGRO MORTAL! Si un usuario malintencionado ingresa como nombre algo como: ' OR '1'='1', podría ejecutar comandos arbitrarios en tu base de datos, borrar información crítica o robar datos sensibles. Este tipo de ataque se conoce como **SQL Injection** y ha sido responsable de algunas de las brechas de seguridad más graves en la historia de la informática.

La Solución Profesional: PreparedStatement

En lugar de concatenar texto directamente, utilizamos marcadores de posición (?) que separan el código SQL de los datos. Java envía los valores por un canal separado, escapando automáticamente cualquier carácter peligroso y haciéndolos completamente inofensivos:

```
String sql = "INSERT INTO usuarios (nombre, edad) VALUES (?, ?)";  
PreparedStatement stmt = con.prepareStatement(sql);  
stmt.setString(1, "Carlos"); // Primer ?  
stmt.setInt(2, 30);          // Segundo ?  
stmt.executeUpdate();
```

Ventaja Extra: La base de datos "pre-compila" la consulta una vez y luego puede reutilizarla múltiples veces con diferentes parámetros, lo que mejora significativamente el rendimiento cuando ejecutas la misma consulta muchas veces con valores diferentes.

PreparedStatement vs Statement

Statement (Inseguro)

- Concatenación directa de strings
- Vulnerable a SQL Injection
- Sin validación automática
- Rendimiento inferior
- Código más propenso a errores

```
String nombre = input;  
String sql = "SELECT * FROM usuarios WHERE nombre='" + nombre + "'";  
stmt.executeQuery(sql);
```

PreparedStatement (Seguro)

- Parámetros separados del SQL
- Inmune a SQL Injection
- Validación y escape automático
- Pre-compilación de consultas
- Código más limpio y mantenible

```
String sql = "SELECT * FROM usuarios WHERE nombre=?";  
PreparedStatement stmt = con.prepareStatement(sql);  
stmt.setString(1, input);  
stmt.executeQuery();
```

El Cursor que Avanza: Entendiendo ResultSet

Cuando ejecutamos un SELECT, la base de datos no nos devuelve directamente una lista de Java (como un ArrayList). En su lugar, nos proporciona un objeto especial llamado **ResultSet**, que funciona como un puntero o cursor que se mueve a través de los registros devueltos.

La Trampa del next() que Todos Cometan

El ResultSet tiene una característica peculiar que confunde a muchos estudiantes: cuando lo obtenemos por primera vez, el cursor está posicionado **antes** del primer registro, en una posición vacía. Muchos intentan leer datos inmediatamente y reciben el temido error: `Before start of result set`.

01

Ejecutar la Consulta

Obtenemos el ResultSet, pero el cursor aún no apunta a ningún registro válido

02

Llamar a next()

Movemos el cursor al primer registro.
Devuelve true si existe, false si no hay datos

03

Leer los Datos

Usamos getString(), getInt(), etc. para extraer los valores de la fila actual

04

Repetir o Finalizar

Si esperamos más registros, volvemos a llamar next() en un bucle while

Patrones de Lectura con ResultSet

Dependiendo de cuántos registros esperemos recuperar de nuestra consulta, utilizaremos diferentes estructuras de control. Es fundamental elegir el patrón correcto para evitar errores y escribir código más expresivo.

Esperamos UN Solo Registro

Cuando sabemos que la consulta debe devolver como máximo un resultado (por ejemplo, buscar por ID único), usamos un **if**:

```
String sql = "SELECT * FROM usuarios  
WHERE id=?";  
PreparedStatement stmt =  
con.prepareStatement(sql);  
stmt.setInt(1, 42);  
ResultSet rs = stmt.executeQuery();  
  
if (rs.next()) {  
    String nombre =  
rs.getString("nombre");  
    int edad = rs.getInt("edad");  
    System.out.println(nombre + " - " +  
edad);  
} else {  
    System.out.println("Usuario no  
encontrado");  
}
```

Esperamos MUCHOS Registros

Cuando queremos procesar todos los registros que coinciden con nuestra consulta, usamos un **while**:

```
String sql = "SELECT * FROM  
productos WHERE categoria=?";  
PreparedStatement stmt =  
con.prepareStatement(sql);  
stmt.setString(1, "Electrónica");  
ResultSet rs = stmt.executeQuery();  
  
while (rs.next()) {  
    String nombre =  
rs.getString("nombre");  
    double precio =  
rs.getDouble("precio");  
    System.out.println(nombre + ": $" +  
precio);  
}  
System.out.println("Listado  
completo");
```

Recuerda que **next()** devuelve un boolean: true si logró moverse a un registro válido, false si ya no quedan más registros por leer. Esta característica es la que permite que el while se detenga automáticamente al terminar.

Transacciones: El Principio de Todo o Nada

Por defecto, JDBC opera en modo **Auto-Commit**. Esto significa que cada operación SQL se guarda permanentemente en la base de datos de forma inmediata, sin posibilidad de retractarse. Para la mayoría de operaciones simples esto funciona perfectamente, pero ¿qué sucede cuando necesitamos ejecutar varias operaciones que deben completarse todas juntas o ninguna?

El Problema de la Consistencia

Imagina una transferencia bancaria entre dos cuentas. El proceso requiere dos operaciones:

1. Restar saldo de la cuenta de Juan
2. Sumar ese mismo saldo a la cuenta de Pedro

Si ocurre un error (corte de luz, excepción, fallo de red) justo después del paso 1 pero antes del paso 2, el dinero literalmente desaparecería del sistema. Juan perdería su dinero pero Pedro nunca lo recibiría. Este tipo de inconsistencia es **inaceptable** en aplicaciones profesionales.

La Solución: Control Manual con Transacciones

Para evitar estos desastres, tomamos control manual del proceso de guardado. Desactivamos el Auto-Commit y agrupamos múltiples operaciones en una **transacción atómica**. El término "atómico" proviene de la palabra griega "átomo" (indivisible): o se ejecuta todo el conjunto de operaciones, o no se ejecuta ninguna.

Implementación de Transacciones en Código

El manejo de transacciones en JDBC sigue un patrón específico que debes memorizar y aplicar en todas tus operaciones críticas. Aquí está el código completo con explicaciones detalladas:

```
Connection con = null;
try {
    con = DataSource.getConnection();

    // 1. APAGAMOS EL PILOTO AUTOMÁTICO
    con.setAutoCommit(false);

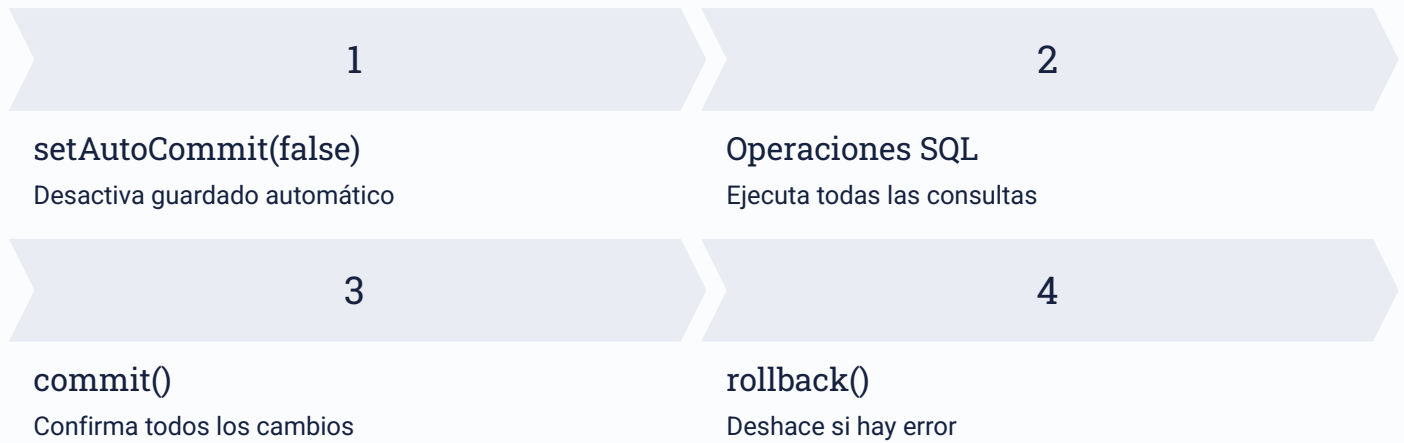
    // 2. EJECUTAMOS TODAS LAS OPERACIONES
    PreparedStatement stmt1 = con.prepareStatement(
        "UPDATE cuentas SET saldo = saldo - ? WHERE id = ?");
    stmt1.setDouble(1, 100.0);
    stmt1.setInt(2, 1);
    stmt1.executeUpdate();

    // Aquí podría ocurrir un error (simulado)
    // int x = 1 / 0; // Descomenta para probar rollback

    PreparedStatement stmt2 = con.prepareStatement(
        "UPDATE cuentas SET saldo = saldo + ? WHERE id = ?");
    stmt2.setDouble(1, 100.0);
    stmt2.setInt(2, 2);
    stmt2.executeUpdate();

    // 3. SI TODO SALIÓ BIEN, CONFIRMAMOS
    con.commit();
    System.out.println("Transferencia completada exitosamente");

} catch (Exception e) {
    // 4. SI ALGO FALLÓ, DESHACEMOS TODO
    if (con != null) {
        try {
            con.rollback();
            System.out.println("Error: Se deshicieron todos los cambios");
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
} finally {
    // 5. RESTAURAMOS EL ESTADO ORIGINAL (¡CRÍTICO!)
    if (con != null) {
        try {
            con.setAutoCommit(true);
            con.close();
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
}
```



El Detalle que Olvidan los Expertos

Hay un paso final en el manejo de transacciones que, si olvidas, puede causar problemas muy difíciles de detectar horas o incluso días después. Este error es especialmente peligroso porque la aplicación seguirá funcionando aparentemente normal, pero los datos simplemente no se guardarán.

El Problema con los Pools de Conexiones

Como aprendimos en la Actividad 1, las aplicaciones profesionales no abren una nueva conexión física a la base de datos cada vez que la necesitan. En su lugar, utilizan un **Pool de Conexiones**: un conjunto de conexiones que se reutilizan entre diferentes usuarios y operaciones.

Cuando terminas de usar una conexión y llamas a `close()`, no estás realmente cerrando la conexión física. Lo que haces es **devolverla al pool** para que otro componente de la aplicación pueda usarla inmediatamente.

Aquí está el peligro: si dejaste esa conexión con `setAutoCommit(false)`, el siguiente usuario que la reciba del pool intentará guardar datos y **no se guardará nada**, porque la conexión sigue esperando un `commit` manual. El programador no entenderá por qué sus `INSERT` o `UPDATE` no funcionan, ya que no verá ningún error.



La Solución: Bloque Finally

Siempre debemos dejar la conexión en el estado en que la encontramos. Usamos un bloque **finally** que se ejecuta sin importar si hubo éxito o error:

```
} finally {  
    if (con != null) {  
        try {  
            con.setAutoCommit(true); // Restaurar estado original  
            con.close(); // Devolver al pool  
        } catch (SQLException ex) {  
            ex.printStackTrace();  
        }  
    }  
}
```

Resumen: Manipulación Profesional de Datos

Hemos completado un recorrido fundamental por las operaciones CRUD con un enfoque profesional que va mucho más allá de simples tutoriales. Estas son las lecciones clave que debes dominar:



Dos Métodos Principales

executeQuery() para SELECT, devuelve ResultSet
executeUpdate() para INSERT, UPDATE, DELETE, devuelve cantidad de filas afectadas



Seguridad Primero

Jamás concatenes Strings en consultas SQL
Siempre usa PreparedStatement con marcadores ? para prevenir SQL Injection



Atomicidad con Transacciones

En operaciones críticas desactiva AutoCommit
Usa commit() para confirmar o rollback() para deshacer cambios



Higiene de Conexiones

Siempre restaura setAutoCommit(true) en el bloque finally
Evita contaminar el pool de conexiones

Próximos Pasos

En la siguiente actividad abandonaremos la práctica de escribir SQL directamente en el método main. Aprenderemos a estructurar nuestro código en capas profesionales utilizando el patrón **DAO (Data Access Object)**, que separa la lógica de acceso a datos de la lógica de negocio. Este es el siguiente nivel en tu camino hacia el desarrollo de aplicaciones empresariales robustas y mantenibles.

"El código profesional no solo funciona, sino que funciona de forma segura, consistente y mantenible. Las transacciones y PreparedStatement no son opcionales: son fundamentales."